

AMAZON E-COMMERCE MANAGEMENT SYSTEM

PAYMENT TABLE

1. DATABASE DESIGN

1.1 PAYMENT TABLE

Attribute	Description
Payment_ID	Unique payment ID
Order_ID	Related order ID
Payment_Date	Date of payment
Payment_Method	UPI / Credit Card / Debit Card / Cash on Delivery
Amount	Amount paid
Payment_Status	Successful / Failed / Pending

2. TABLE CREATION

The PAYMENT table is created to store payment-related information for customer orders.

```
SQL> CREATE TABLE PAYMENT (  
  2  PAYMENT_ID NUMBER PRIMARY KEY,  
  3  ORDER_ID NUMBER NOT NULL,  
  4  PAYMENT_DATE DATE NOT NULL,  
  5  PAYMENT_METHOD VARCHAR2(30) NOT NULL,  
  6  PAYMENT_STATUS VARCHAR2(20) NOT NULL,  
  7  AMOUNT NUMBER(10,2) NOT NULL  
  8  );
```

Table created.

The PAYMENT table is successfully created with attributes for payment ID, order ID, payment date, payment method, payment status, and amount.

3. VALUE INSERTION

Payment details are inserted into the PAYMENT table using the following SQL commands.

```

SQL> INSERT INTO PAYMENT
  2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS, AMOUNT)
  3 VALUES
  4 (1, 1, TO_DATE('20-09-2026','DD-MM-YYYY'), 'UPI', 'Completed', 1200.00);

1 row created.

SQL> INSERT INTO PAYMENT
  2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS, AMOUNT)
  3 VALUES
  4 (2, 2, TO_DATE('20-09-2026','DD-MM-YYYY'), 'Credit Card', 'Completed', 850.00);

1 row created.

SQL> INSERT INTO PAYMENT
  2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS, AMOUNT)
  3 VALUES
  4 (3, 3, TO_DATE('21-09-2026','DD-MM-YYYY'), 'Debit Card', 'Completed', 1500.00);

1 row created.

SQL> INSERT INTO PAYMENT
  2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS, AMOUNT)
  3 VALUES
  4 (4, 4, TO_DATE('21-09-2026','DD-MM-YYYY'), 'Cash on Delivery', 'Pending', 650.00);

1 row created.

SQL>
SQL> INSERT INTO PAYMENT
  2 (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_METHOD, PAYMENT_STATUS, AMOUNT)
  3 VALUES
  4 (5, 5, TO_DATE('21-09-2026','DD-MM-YYYY'), 'UPI', 'Completed', 2200.00);

```

4. DISPLAY SUCCESSFUL PAYMENTS

Successful payment transactions are displayed using the following query:

5. DISPLAY FAILED / PENDING PAYMENTS

Failed or pending transactions can be displayed using:

```
SQL> SELECT
  2     PAYMENT_ID,
  3     ORDER_ID,
  4     PAYMENT_METHOD,
  5     PAYMENT_DATE,
  6     PAYMENT_STATUS,
  7     AMOUNT
  8 FROM PAYMENT;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D
1	1	UPI	20-SEP-26
Successful		1200	
2	2	Credit Card	20-SEP-26
Successful		850	
3	3	Debit Card	21-SEP-26
Successful		1500	
4	4	Cash on Delivery	21-SEP-26
Successful		650	
5	5	UPI	21-SEP-26
Successful		2200	

This query displays all payment transactions whose status is marked as Successful

5. DISPLAY FAILED / PENDING PAYMENTS

Failed or pending transactions can be displayed using:

```
SQL> SELECT *
  2 FROM PAYMENT
  3 WHERE PAYMENT_STATUS = 'Failed';

no rows selected

SQL>
```

6. MODIFY PAYMENT STATUS

The payment status can be modified when the transaction status changes.

```
SQL> SELECT
  2     PAYMENT_METHOD,
  3     COUNT(*) AS TOTAL_TRANSACTIONS
  4 FROM PAYMENT
  5 GROUP BY PAYMENT_METHOD
  6 ORDER BY TOTAL_TRANSACTIONS DESC;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS
UPI	2
Debit Card	1
Cash on Delivery	1
Credit Card	1

7. ANALYZE PAYMENT METHODS

Payment methods used by customers can be analyzed using SQL aggregation queries.

7.1 COUNT TRANSACTIONS BY PAYMENT METHOD

```
SQL> SELECT
  2     PAYMENT_METHOD,
  3     COUNT(*) AS TOTAL_TRANSACTIONS
  4 FROM PAYMENT
  5 GROUP BY PAYMENT_METHOD
  6 ORDER BY TOTAL_TRANSACTIONS DESC;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS
UPI	2
Debit Card	1
Cash on Delivery	1
Credit Card	1

```
SQL> |
```

This query calculates the number of transactions made using each payment method.

7.2 TOTAL AMOUNT BY PAYMENT METHOD

```
SQL> SELECT
2     PAYMENT_METHOD,
3     COUNT(*) AS TOTAL_TRANSACTIONS,
4     SUM(AMOUNT) AS TOTAL_AMOUNT
5 FROM PAYMENT
6 WHERE PAYMENT_STATUS = 'Successful'
7 GROUP BY PAYMENT_METHOD
8 ORDER BY TOTAL_AMOUNT DESC;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS	TOTAL_AMOUNT
UPI	2	3400
Debit Card	1	1500
Credit Card	1	850
Cash on Delivery	1	650

8. PAYMENT TRANSACTION REPORT

A complete payment transaction report can be generated by joining the PAYMENT and ORDERS tables

```
SQL> SELECT
2     P.PAYMENT_ID,
3     P.ORDER_ID,
4     O.CUSTOMER_ID,
5     P.PAYMENT_METHOD,
6     P.PAYMENT_DATE,
7     P.PAYMENT_STATUS,
8     P.AMOUNT
9 FROM PAYMENT P
10 JOIN ORDERS O
11 ON P.ORDER_ID = O.ORDER_ID
12 ORDER BY P.PAYMENT_DATE;
```

PAYMENT_ID	ORDER_ID	CUSTOMER_ID	PAYMENT_METHOD	PAYMENT_D
2	2	2	Credit Card	20-SEP-26
Successful		850		
1	1	1	UPI	20-SEP-26
Successful		1200		
4	4	1	Cash on Delivery	21-SEP-26
Successful		650		
5	5	2	UPI	21-SEP-26
Successful		2200		

10. FINAL PAYMENT TABLE

The final PAYMENT table contains the following records:

Payment_ID	Order_ID	Payment_Date	Payment_Method	Payment_Status	Amount
1	1	20-SEP-2026	UPI	Successful	1200.00
2	2	20-SEP-2026	Credit Card	Successful	850.00
3	3	21-SEP-2026	Debit Card	Successful	1500.00
4	4	21-SEP-2026	Cash on Delivery	Failed	650.00
5	5	21-SEP-2026	UPI	Successful	2200.00

11. CONCLUSION

The Payment Management module successfully manages customer payment transactions in the e-commerce database. The PAYMENT table stores payment details such as payment method, date, status, and amount. SQL queries are used to manage successful and failed transactions, analyze payment methods, and generate payment transaction and customer-wise reports. This module helps maintain organized payment information and supports efficient transaction management.